# 2.19 Real stuff: ARMv8 (64-bit) instruction set

The figure below suggests the size of the full ARMv8 architecture by listing the number of assembly language instructions and machine language instructions in each class of instruction types. COD Chapter 3 (Arithmetic for Computers) describes the floating point, multiply-divide, and SIMD portion of the architecture and COD Chapter 5 (Large and Fast: Exploiting Memory Hierarchy) covers the system instructions. In this section, we highlight the integer operations and branches.

Figure 2.19.1: The number of instructions of each type and for each function, counted separately for assembly language instructions (AL) and machine language instructions (ML) (COD Figure 2.40).

The counts were based on evaluating the *ARM Architecture Reference Manual: ARMv8, for ARMv8-A architecture profile*, beta edition, 2014. There were also a few minor adjustments made to match the version of the assembly language instructions in this book. For example, this book shows different assembly language instructions for single and precision floating-point arithmetic (`FADDS, FADDD,`…) but ARMv8 actually uses just one version (`FADD`), using the register names to select the opcode.

| Class | Loads/Stores | | Operations | | Branches | | Total | |
|---|---|---|---|---|---|---|---|---|
| | AL | ML | AL | ML | AL | ML | AL | ML |
| Integer | 49 | 145 | 74 | 105 | – | – | 123 | 250 |
| Floating Point & Int Mul/Div | 0 | 18 | 63 | 156 | – | – | 63 | 174 |
| SIMD/Vector | 16 | 166 | 229 | 371 | – | – | 245 | 537 |
| System/Special | 11 | 55 | 52 | 40 | – | – | 63 | 95 |
| – | – | – | – | – | 23 | 14 | 23 | 14 |
| Total | 76 | 384 | 418 | 672 | 23 | 14 | 517 | 1070 |

One reason for the much larger number of machine instructions than assembly instructions is that ARMv8 includes both 32-bit and 64-bit versions of instructions within the same architecture. In assembly language, programmers use registers named `W0, W1,`… instead of the `X0, X1,`… to specify 32-bit operations. Thus, this 64-bit operation

```
ADD    X9, X21, X9
```

can be turned into a 32-bit operation by writing instead

```
ADD    W9, W21, W9
```

The figure above counts this as one assembly language instruction but two machine language instructions since they have different opcodes.

Our goal for the rest of this section is to explain any remaining instructions in ARMv8. It is also to give insight into how the 36 LEGv8 instructions in COD Figure 2.1 (LEGv8 assembly language revealed…) increase to 146 in the full ARMv8 instruction set: 123 assembly language instructions in the first row in The figure above plus 23 branch instructions in the second to last row.

**PARTICIPATION ACTIVITY** 2.19.1: ARMv8 assembly and machine language instructions.

How to use this tool

245 | W0, W1, .... | 250

| | |
|---|---|
| | Total number of ARMv8 integer machine language instructions. |
| | Total number of ARMv8 SIMD/Vector assembly language instructions. |
| | ARMv8 assembly language registers used for 32-bit operations. |

Reset

## Full ARMv8 integer arithmetic logic instructions

A unique feature of the ARMv8 instruction set is that the second register of all arithmetic and logical processing operations has the option of being shifted before being operated on. The shift options are shift left logical, shift right logical, shift right arithmetic, and rotate right. (For the 12-bit immediate instructions, the only shift option is a logical left shift by 12 bits.) Although the assembler has explicit instructions with these names (`LSL`, `LSR`, `SRA`, and `ROR`), these are really just pseudoinstructions of a regular instruction that only performs the shift . We picked versions of the instructions that put 0 in the shift amount field so that the second operand is unchanged in the LEGv8, but the full instruction set allows shifts as part of arithmetic and logic instructions.

The figure below lists all 74 assembly language instructions for the integer operations of the full ARMv8 instruction set. We start with the 20 integer operations in the LEGv8 in COD Figure 2.1 (LEGv8 assembly language revealed…). First, the total count of assembly language instructions includes pseudoinstructions from above—`CMP`, `CMPI`, `MOV`—so that bumps the count to 23. There are also five pseudoinstructions based on the instructions in the LEGv8—negate, negate and set flags, compare negative, compare negative immediate, and test immediate—which brings us to 28.

Figure 2.19.2: The list of assembly language instructions for the integer operations in the full ARMv8 instruction set (COD Figure 2.41).

Bold means the instruction is also in LEGv8, italic means it is a pseudoinstruction, and bold italic means it is a pseudoinstruction that is also in LEGv8.

| Type | Mnemonic | Instruction | Type | Mnemonic | Instruction |
|---|---|---|---|---|---|
| Arithmetic Register | **ADD** | Add | Logical Immediate | **ANDI** | Bitwise AND Immediate |
| | **ADDS** | Add and set flags | | **ANDIS** | Bitwise AND and set flags Immediate |
| | **SUB** | Subtract | | **ORRI** | Bitwise inclusive OR Immediate |
| | **SUBS** | Subtract and set flags | | **EORI** | Bitwise exclusive OR Immediate |
| | ***CMP*** | Compare | | *TSTI* | Test bits Immediate |
| | *CMN* | Compare negative | Shift Register Shift Immed | **LSL** | Logical shift left Immediate |
| | *NEG* | Negate | | **LSR** | Logical shift right Immediate |
| | *NEGS* | Negate and set flags | | ASR | Arithmetic shift right Immediate |
| Arithmetic Immediate | **ADDI** | Add Immediate | | ROR | Rotate right Immediate |
| | **ADDIS** | Add and set flags Immediate | | LSRV | Logical shift right register |
| | **SUBI** | Subtract Immediate | | LSLV | Logical shift left register |
| | **SUBIS** | Subtract and set flags Immediate | | ASRV | Arithmetic shift right register |
| | **CMPI** | Compare Immediate | | RORV | Rotate right register |
| | *CMNI* | Compare negative Immediate | Move Wide Immed iate | **MOVZ** | Move wide with zero |
| Arithmetic Extended | ADD | Add Extended Register | | **MOVK** | Move wide with keep |
| | ADDS | Add and set flags Extended | | MOVN | Move wide with NOT |
| | SUB | Subtract Extended Register | | ***MOV*** | Move register |
| | SUBS | Subtract and set flags Extended | Bit Field Insert & Extract | BFM | Bitfield move |
| | *CMP* | Compare Extended Register | | SBFM | Signed bitfield move |
| | *CMN* | Compare negative Extended | | UBFM | Unsigned bitfield move (32-bit) |
| Arithmetic with Carry | ADC | Add with carry | | BFI | Bitfield insert |
| | ADCS | Add with carry and set flags | | BFXIL | Bitfield extract and insert low |
| | SBC | Subtract with carry | | SBFIZ | Signed bitfield insert in zero |
| | SBCS | Subtract with carry and set flags | | SBFX | Signed bitfield extract |
| | *NGC* | Negate with carry | | UBFIZ | Unsigned bitfield insert in zero |
| | *NGCS* | Negate with carry and set flags | | UBFX | Unsigned bitfield extract |
| Logical Register | **AND** | Bitwise AND | | EXTR | Extract register from pair |
| | **ANDS** | Bitwise AND and set flags | Sign Extend | *SXTB* | Sign-extend byte |
| | **ORR** | Bitwise inclusive OR | | *SXTH* | Sign-extend halfword |
| | **EOR** | Bitwise exclusive OR | | *SXTW* | Sign-extend word |
| | BIC | Bitwise bit clear | | *UXTB* | Unsigned extend byte |
| | BICS | Bitwise bit clear and set flags | | *UXTH* | Unsigned extend halfword |
| | ORN | Bitwise inclusive OR NOT | Bit Operation | CLS | Count leading sign bits |
| | EON | Bitwise exclusive OR NOT | | CLZ | Count leading zero bits |
| | *MVN* | Bitwise NOT | | RBIT | Reverse bit order |
| | *TST* | Test bits | | REV | Reverse bytes in register |
| | | | | REV16 | Reverse bytes in halfwords |
| | | | | REV32 | Reverses bytes in words |

To support arithmetic on narrower data types, there are another six instructions that let you mix data sizes of the second operand by either sign extending it or zero extending it to the full width. The *extended-register* instructions work with bytes, halfwords, or words, which gets us to 34 assembly language instructions.

To support add and subtract operations on operands larger than one doubleword, ARM includes six more assembly instructions to add or subtract the carry from a previous operation. Thus, the first instruction does the regular add of the registers with the right half of the operands and sets the condition codes (`ADDS`), and the second one does add with the carry out from that operation when adding the registers with the left half of the operands (`ADC`). These six bring the total to 40 instructions.

The logical instructions (`AND`, `ANDS`, `ORR`, `EOR`) also have versions where the second operand is complemented: that is, every 0 becomes 1 and vice versa. Including new pseudoinstructions, this variation leads to six more instructions, or a total now of 46.

In addition to the logical shift left and logical shift right instructions, ARMv8 includes two more, as mentioned above. `ASR` does arithmetic shift right, which replicates the sign bit during the shift, and `ROR` rotates the bits to the right; that is, the bits shift off to the right are inserted on the left. Although we didn't list them in LEGv8, there are versions of all the shift instructions that determine the amount to be shifted based on a value in a register rather than as an immediate within the instruction. These shifts add six more instructions, bringing us to 52.

There is also a version of the `MOV` wide instructions (`MOVN`) that complements all 64 bits that are created from the 16-bit constant; that is, the other 48 bits are ones instead of zeros and the 16-bit immediate field is complemented too, which bumps the count to 53.

To manipulate fields of bits, the full ARMv8 instruction set includes 10 instructions that can extract a bit field from a register and insert it into another. It uses those instructions to create five more pseudoinstructions that sign extend or zero extend narrow operands to the full width of a register. Finally, there are another six "bit twiddling" instructions that count leadings zeros or ones and reverse order of bits and bytes. The total is now 74 integer operations in the full ARMv8 integer operation assembly language.

**PARTICIPATION ACTIVITY** 2.19.2: ARMv8 integer arithmetic and logic assembly language instructions.

Select the correct instruction type for each instruction.

1) `CMP`

- ○ Arithmetic
- ○ Logical

2) `MOV`

- ○ Shift
- ○ Move wide

3) `LSL`

- Logical
- Shift

4) `ANDS`

- Logical
- Move wide

## Full ARMv8 integer data transfer instructions

We did not show all of the addressing modes available for the 10 integer data transfer instructions in LEGv8. The addressing mode we described earlier is called *unscaled signed immediate offset* in ARMv8 lingo. Here are five more:

1. Base plus a scaled 12-bit unsigned immediate offset.
2. Base plus a 64-bit register offset, optionally scaled.
3. Base plus a 32-bit extended register offset, optionally scaled.
4. Pre-indexed by an unscaled 9-bit signed immediate offset.
5. Post-indexed by an unscaled 9-bit signed immediate offset.

The scaling options of the first three addressing modes multiply or scale the address in the immediate field or in the register by the size of the data being transferred in bytes. Since you would normally want that address to be a multiple of the data size, these addressing modes are built into the hardware so the programmer doesn't have to convert an index in a register into a byte address. They also increase the range of what the immediate field can represent within an instruction. Thus, if `X11` contains $100{,}000_{ten}$

```
LDR    X10, [X11, #16]          // scaled addressing mode
```

will load the double word (8 bytes) at address $100{,}128_{ten}$ (100,000 + 8*16) into register `X10`.

The address of the second addressing mode is simply the sum of two registers, with the option of shifting the second operand by 1, 2, or 3 bits, which multiplies the value by 2, 4, or 8. Thus, if `X11` contains $100{,}000_{ten}$ and `X12` contains $1{,}000_{ten}$

```
LDR    X10, [X11, X12 LSL #3]  // base + register, scaled
```

will load the double word (8 bytes) at address $108{,}000_{ten}$ (100,000 + $2^3$*1000) into register `X10`. The third addressing mode simply uses a 32-bit register (e.g., `W12`) instead of a 64-bit register (e.g., `X12`), with the same option of scaling.

The last two addressing modes *change the base register* as part of the address calculation. The use case is when marching through a sequential array, the address increment can happen as part of the

addressing mode rather than as part of a separate add or subtract instruction. Thus, if `X11` contains $100{,}000_{ten}$

```
LDR    X10, [X11, #16]!        // pre-indexed addressing mode
```

will load the double word (8 bytes) at address $100{,}016_{ten}$ into register `X10` and change `X11` to $100{,}016_{ten}$. On the other hand,

```
LDR    X10, [X11], #16         // post-indexed addressing mode
```

will load the double word (8 bytes) at address $100{,}000_{ten}$ into register `X10` and change `X11` to $100{,}016_{ten}$. The pre- and post- name indicates whether the address arithmetic occurs before the operand is accessed or afterwards.

The main reason there are so many more machine data transfer instructions (125) than assembly data transfer instructions (49) for integers is that the addressing modes require separate opcodes in machine language, but as we see above, they are just a different addressing notation for the same mnemonics in assembly language.

The figure below lists the full set of integer data transfer instructions for ARMv8. To get from 10 integer data-transfer instructions to the ARMv8 total of 49, we first add the `LDA` pseudoinstruction, to get us to 11. Next, we note that there are signed data transfer versions of the *load byte* (`LDURSB`) and *load halfword* (`LDURSH`), which brings us to 13. In addition to the doubleword load exclusive and store exclusive instructions, ARMv8 includes exclusive data transfers of bytes, halfwords, and paired double words, yielding another six instructions, or a total of 19.

Figure 2.19.3: The list of assembly language instructions for the integer data transfer operations in the full ARMv8 instruction set (COD Figure 2.42).

Bold means the instruction is also in LEGv8, italic means it is a pseudoinstruction, and bold italic means it is a pseudoinstruction that is also in LEGv8.

| Type | Mnemonic | Instruction | Type | Mnemonic | Instruction |
|---|---|---|---|---|---|
| Unscaled | **LDUR** | Load register (unscaled offset) | Exclusive | **LDXR** | Load Exclusive register |
| | **LDURB** | Load byte (unscaled offset) | | LDXRB | Load Exclusive byte |
| | LDURSB | Load signed byte (unscaled offset) | | LDXRH | Load Exclusive halfword |
| | **LDURH** | Load halfword (unscaled offset) | | LDXP | Load Exclusive Pair |
| | LDURSH | Load signed halfword (unscaled offset) | | **STXR** | Store Exclusive register |
| | **LDURSW** | Load signed word (unscaled offset) | | STXRB | Store Exclusive byte |
| | **STUR** | Store register (unscaled offset) | | STXRH | Store Exclusive halfword |
| | **STURB** | Store byte (unscaled offset) | | STXP | Store Exclusive Pair |
| | **STURH** | Store halfword (unscaled offset) | Exclusive Aquire/Release | LDAXR | Load-aquire Exclusive register |
| | **STURW** | Store word (unscaled offset) | | LDAXRB | Load-aquire Exclusive byte |
| | ***LDA*** | Load address | | LDAXRH | Load-aquire Exclusive halfword |
| Scaled, Extended, Pre-& Post-Indexed | LDR | Load register | | LDAXP | Load-aquire Exclusive Pair |
| | LDRB | Load byte | | STLXR | Store-release Exclusive register |
| | LDRSB | Load signed byte | | STLXRB | Store-release Exclusive byte |
| | LDRH | Load halfword | | STLXRH | Store-release Exclusive halfword |
| | LDRSH | Load signed halfword | | STLXP | Store-release Exclusive Pair |
| | LDRSW | Load signed word | Pair | LDP | Load Pair |
| | STR | Store register | | LDPSW | Load Pair signed words |
| | STRB | Store byte | | STP | Store Pair |
| | STRH | Store halfword | PC | ADRP | Compute address of 4KB page at a PC-relative offset |
| | | | | ADR | Compute address of label at a PC-relative offset |

The data transfers with the other addressing modes use a different mnemonic (`LDR, STR,`…), which leads to another nine instructions to get the various sizes both signed and unsigned. To accelerate data transfers, ARMv8 includes three load pair and store pair instructions, which transfer two doublewords at a time. To create PC-relative addresses that can be passed as arguments, ARMv8 has two instructions that add a 21-bit immediate in the current instruction to the current value of the PC and then load it (or the value shifted left by 12 bits) into a register. These 14 instructions bring us to 33.

The final set of data transfer instructions for integers perform exclusive access to memory in multiprocessor environments. We saw two examples earlier in `LDXR` and `STXR`. In addition to providing exclusive access to doublewords, there are versions for bytes, halfwords, and paired registers. ARMv8 also offers load acquire and store release versions, which implement the release-consistency memory model to support programming language standards for shared memory semantics. This set adds 16 exclusive data transfer instructions in the second column of the figure above, bringing us to the final total of 49 data transfer instructions.

**PARTICIPATION ACTIVITY** 2.19.3: ARMv8 integer data transfer instructions.

1) `LDR X10, [X11, #16]!`

- Pre-indexed addressing mode
- Post-indexed addressing mode

2) Number of machine data transfer instructions for integers

- 125
- 49

3) Meaning of `LDXR`

- Load address
- Load exclusive register

## Full ARMv8 branch instructions

The figure below lists all the branch instructions for ARMv8. To get from six branch instructions in LEGv8 to the ARMv8 total of 23, we start with two more conditional branch instructions that test to see if a bit within a register is zero or one. There are also two more unconditional branches. The first is a variation of branch and link that uses a register for the *branch address* (`BALR`). The second is return from *subroutine* (`RET`), which sounds a lot like *branch register* (`BR`); the reason they sound similar is that they have the identical semantics. The reason ARMv8 has different opcodes for the same operation is so that hardware branch predictors (COD Chapter 4 (The Processor)) can know whether it is really return from a *subroutine* (`RET`), which is easy to predict, or being used in a *branch table* (`BR`), which is much harder to predict and should use a distinct mechanism. These additions get us to 10 branches.

Figure 2.19.4: The list of assembly language instructions for the branches of the ARMv8 instruction set (COD Figure 2.43).

Bold means the instruction is also in LEGv8, italic means it is a pseudoinstruction, and bold italic means it is a pseudoinstruction that is also in LEGv8.

| Type | Mnemonic | Instruction | Type | Mnemonic | Instruction |
|---|---|---|---|---|---|
| Conditional Branch | **B.cond** | Branch conditionally | Conditional Select | CSEL | Conditional select |
| | **CBNZ** | Compare and branch if nonzero | | CSINC | Conditional select increment |
| | **CBZ** | Compare and branch if zero | | CSINV | Conditional select inversion |
| | TBNZ | Test bit and branch if nonzero | | CSNEG | Conditional select negation |
| | TBZ | Test bit and branch if zero | | *CSET* | Conditional set |
| Unconditional Branch | **B** | Branch unconditionally | | *CSETM* | Conditional set mask |
| | **BL** | Branch with link | | *CINC* | Conditional increment |
| | BLR | Branch with link to register | | *CINV* | Conditional invert |
| | **BR** | Branch to register | | *CNEG* | Conditional negate |
| | RET | Return from subroutine | Conditional Compare | CCMP | Conditional compare register |
| | | | | CCMPI | Conditional compare immediate |
| | | | | CCMN | Conditional compare negative register |
| | | | | CCMNI | Conditional compare negative immediate |

The next nine instructions store a value into a register based on the condition codes. If the condition is true, the destination register gets the first register. If not, it gets the second register. The idea behind condition select instructions is to replace conditional branches, which can cause problems in pipelined execution if they can't be predicted (see COD Chapter 4 (The Processor)). After adding the nine condition select instructions, we're up to 19.

The final four instructions are similar to the conditional select instructions, except the destination for these instructions is the condition codes. That is, these instructions conditionally set the condition codes based on the current condition codes, which is a bit confusing. If the selected condition is true, then the condition codes are set by the result of an arithmetic comparison of two operands, or to an immediate value if the condition is false. One version of conditional compare instruction checks two registers, and the other compares a register to a small 5-bit unsigned value. These last four instructions raise the total to 23 branch instructions in ARMv8.

**PARTICIPATION ACTIVITY** 2.19.4: ARMv8 branch instructions.

How to use this tool

23 BR RET 6

| | |
|---|---|
| | Number of branch instructions in LEGv8 |
| | Return from subroutine instruction |
| | Number of branch instructions in ARMv8 |
| | Branch register instruction |

Reset